The Journal of Systems & Software

ARTEMIS: Testing Internal Structure of Agent Prompts in Multi-Agent Systems

--Manuscript Draft--

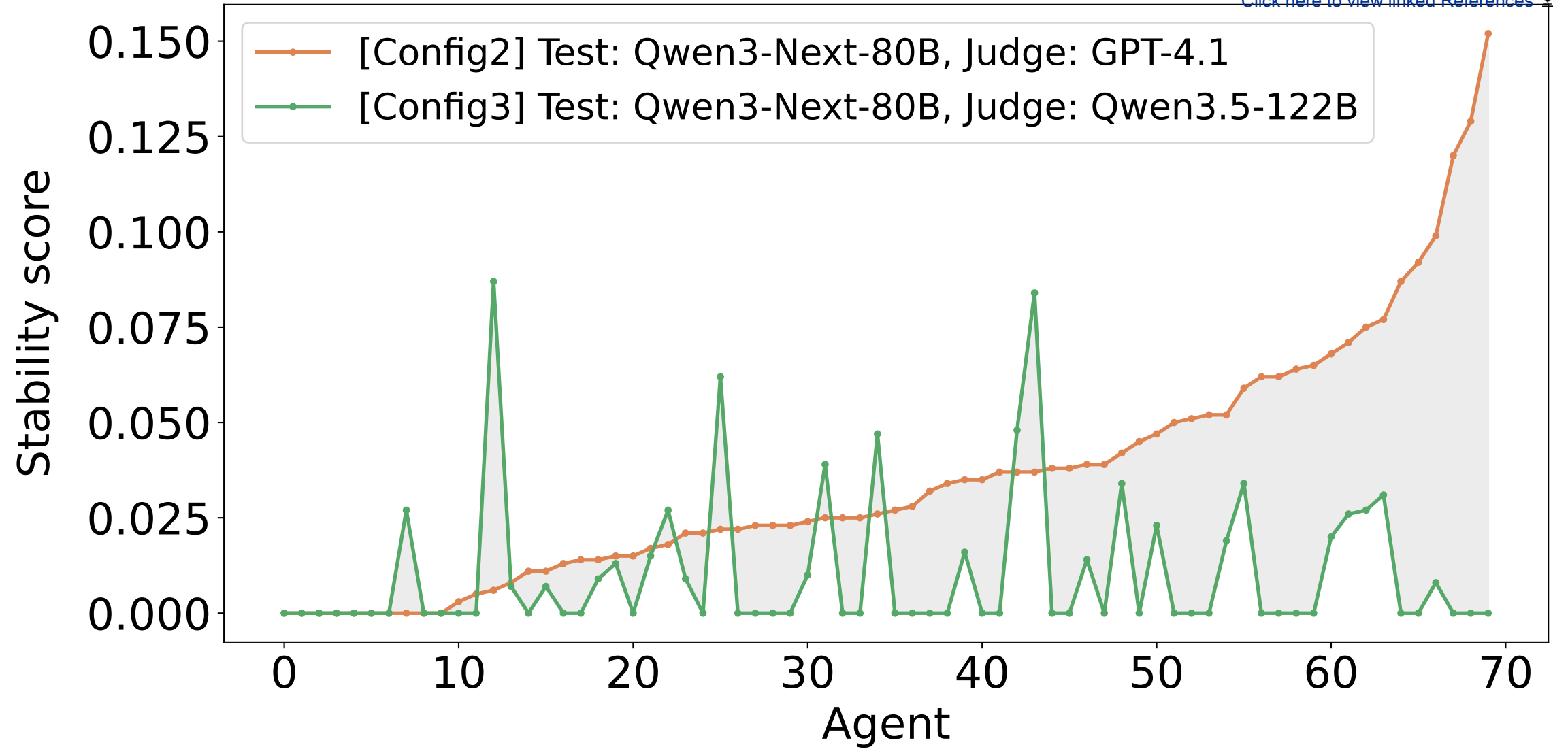
Manuscript Number:	
Article Type:	VSI:SQA4AI-JSS
Keywords:	Prompt testing; multi-agent system; large language model; LLM-as-a-judge
Corresponding Author:	Duc-Anh Nguyen VNU University of Engineering and Technology (VNU-UET) VIET NAM
First Author:	Hiep Pham Hoang
Order of Authors:	Hiep Pham Hoang
	Dat Nguyen Trong
	Hoang-Viet Tran
	Duc-Anh Nguyen
	Pham Ngoc Hung
Abstract:	Multi-Agent Systems (MAS) based on Large Language Models (LLMs) are increasingly deployed, but their quality assurance remains a challenge. A system prompt could be complicated and contain redundant descriptions. Existing methods only test the output of the agents from a system prompt without considering the internal structure of this system prompt. Therefore, this paper presents ARTEMIS to test the internal structure of system prompts. For each agent, ARTEMIS decomposes its system prompt into a set of requirements, then generates test cases to discover all requirements. For each test case, ARTEMIS runs the agent repeatedly, and scores every response with a judge LLM. To evaluate an agent, ARTEMIS proposes accuracy and instability metrics. While the accuracy score evaluates the extent to which the agent fulfills the requirements, the instability score indicates whether the agent produces consistent results when repeatedly run on the same user queries. To demonstrate the effectiveness of ARTEMIS, we evaluate ARTEMIS on a set of agents from sixteen open-source LangGraph systems. The experiment shows that there are a few well-designed system prompts, while most system prompts contain problems. The three most common failure types are (1) an unclear system prompt (36.3%), (2) an attempt by the user query to violate the rule defined in the system prompt (37.1%), and (3) an incomplete response from the agent (22.3%). These observations demonstrate the effectiveness of the proposed method for evaluating the reliability of the system prompts. Additionally, they shed light on the improvement of the system prompts in future work.

Powered by Editorial Manager® and ProduXion Manager® from Aries Systems Corporation

Highlight

- Proposes ARTEMIS to test the internal structure of agent system prompts.
- Introduces accuracy and stability metrics to assess the reliability of agents.
- Evaluated on 71 agents from 16 open-source LangGraph multi-agent systems.
- Identifies the three most common failure types: (1) an unclear system prompt, (2) an attempt by the user prompt to violate the rule defined in the system prompt, and (3) an incomplete response from the agent.
- Based on the accuracy and stability scores, developers could enhance the system prompts.

Click here to view linked References



Click here to view linked References

ARTEMIS: Testing Internal Structure of Agent Prompts in Multi-Agent Systems

Hiep Pham Hoang, Dat Nguyen Trong, Hoang-Viet Tran, Duc-Anh Nguyen*, Pham Ngoc Hung

VNU University of Engineering and Technology (VNU-UET), Hanoi, Vietnam

Abstract

Multi-Agent Systems (MAS) based on Large Language Models (LLMs) are increasingly deployed, but their quality assurance remains a challenge. A system prompt could be complicated and contain redundant descriptions. Existing methods only test the output of the agents from a system prompt without considering the internal structure of this system prompt. Therefore, this paper presents **ARTEMIS** to test the internal structure of system prompts. For each agent, **ARTEMIS** decomposes its system prompt into a set of requirements, then generates test cases to discover all requirements. For each test case, **ARTEMIS** runs the agent repeatedly, and scores every response with a judge LLM. To evaluate an agent, **ARTEMIS** proposes accuracy and instability metrics. While the accuracy score evaluates the extent to which the agent fulfills the requirements, the instability score indicates whether the agent produces consistent results when repeatedly run on the same user queries. To demonstrate the effectiveness of **ARTEMIS**, we evaluate **ARTEMIS** on a set of agents from sixteen open-source LangGraph systems. The experiment shows that there are a few well-designed system prompts, while most system prompts contain problems. The three most common failure types are (1) an unclear system prompt (36.3%), (2) an attempt by the user query to violate the rule defined in the system prompt (37.1%), and (3) an incomplete response from the agent (22.3%). These observations demonstrate the effectiveness of the proposed method for evaluating the reliability of the system prompts. Additionally, they shed light on the improvement of the system prompts in future work.

Keywords: Prompt testing, multi-agent system, large language model, LLM-as-a-judge

1. Introduction

Multi-Agent Systems (MAS) based on Large Language Models (LLMs) have become popular for handling problems that a single model struggles to solve. In these systems, several agents are arranged into a workflow, each with a role defined by a system prompt (Guo et al., 2024; Wang et al., 2024; Luo et al., 2025; Tran et al., 2025). Frameworks such as MetaGPT (Hong et al., 2024) and ChatDev (Qian et al., 2024) show that this design can outperform a single model on complex tasks.

System prompt of an agent plays an important role in ensuring a reliable agent. The system prompt defines the role, tasks, rules, and behavioral constraints of each agent (Choi et al., 2025; Zhang et al., 2025; Neumann et al., 2025). For example, consider an agent responsible for automatically responding to emails. The role of the agent is to act as a customer support representative. The task of the agent is to read the email content and compose an appropriate response. The rule is that the agent must use a polite tone and only use the provided information. The agent has behavioral constraints that prohibit it from fabricating information or providing information beyond the permitted scope.

*Corresponding author.

Email addresses: 22028005@vnu.edu.vn (Hiep Pham Hoang), 24025033@vnu.edu.vn (Dat Nguyen Trong), thv@vnu.edu.vn (Hoang-Viet Tran), nguyenducanh@vnu.edu.vn (Duc-Anh Nguyen), hungpn@vnu.edu.vn (Pham Ngoc Hung)

However, the system prompt of an agent could be designed ineffectively by the developers. This system prompt could contain unclear descriptions, redundant sentences, etc. (Tian et al., 2025; Yang et al., 2026; Zamfirescu-Pereira et al., 2023). An example of an unclear description is using the term *done*, *complete*, but the developers do not explain the criteria of these terms sufficiently. In this case, the agent may use its own understanding to decide these terms. Finally, if an agent is not well-designed, a small flaw in one agent can quietly cascade through the whole system (Pan et al., 2025; Jamshidi et al., 2026; Dong et al., 2025).

Additionally, a bad system prompt can easily lead to unstable outputs. Specifically, considering an agent, the same user queries can produce different outputs due to non-determinism of the LLMs (Hasan et al., 2026; Atil et al., 2024; Yagubyan, 2026; Mohammadi et al., 2025). An agent with low instability tends to produce consistent response semantics across trials given the same user queries. In contrast, if the response of an agent varies significantly given the same user queries, the instability of this agent is high, leading to low confidence in the response.

To evaluate the reliability of a system prompt, common approaches include using LLM-as-a-judge and prompt attacks. Firstly, the LLM-as-a-judge approach utilizes an independent LLM to evaluate the output of the agent based on user-defined criteria. This approach primarily proposes metrics and methods for the judge model to score objectively. Rep-

resentative methods include DeepEval (Confident AI, 2024), Arize Phoenix[1], LangSmith[2], OpenAI Evals[3], AgentEvals[4], AutoChecklist (Uniyal et al., 2026), G-Eval (Liu et al., 2023), Prometheus (Kim et al., 2024). Secondly, the prompt attack approach generates adversarial user queries to induce incorrect responses from the agent, such as Promptbench (Zhu et al., 2024) and InspectAI[5].

To the best of our knowledge, there is **no standardized method for evaluating the internal structure of system prompts**. We suggest that each component inside the system prompt should be tested by at least one test case. Therefore, this paper proposes **ARTEMIS**, a method that tests each agent of a LangGraph-based MAS in isolation. The input is the source code of a MAS. The output is a per-agent report of accuracy and instability metrics. The **accuracy score** measures how well the agent satisfies the requirements, where each requirement is a portion of the system prompt. The **instability score** measures the inconsistency of its results across repeated runs with the same user queries. From the system prompt of an agent, **ARTEMIS** (1) decomposes its system prompt into individual requirements, (2) generates test cases from the requirements, (3) runs the agent several times per test case, and (4) scores the responses with a LLM-as-a-judge.

To demonstrate the effectiveness of the proposed method, the experiment is conducted on a set of agents extracted from sixteen open-source LangGraph systems. The experiment shows that most system prompts contain problems. The three most common failure types are (1) an unclear system prompt (36.3%), (2) an attempt by the user query to violate the rule defined in the system prompt (37.1%), and (3) an incomplete response from the agent (22.3%). Based on these failures, the developers could adjust the system prompts to enhance its accuracy and instability.

Briefly, the contribution of the paper is as follows:

- **Testing the internal structure of system prompts**: We suggest that testing the internal structure of system prompts is necessary to ensure that the agents are reliable.
- **Metric**: We propose accuracy and instability metrics to capture how well an agent meets its system prompt and how inconsistent it is across repeated runs on the same user queries, respectively.
- **An automated agent-testing pipeline**: Our idea is integrated into a pipeline, which extracts each agent and its system prompt directly from source code and tests it end to end.
- **An empirical evaluation**: We conduct experiments on sixteen open-source LangGraph systems and release the source code for community research.

[1]https://arize.com/phoenix/
[2]https://www.langchain.com/langsmith-platform
[3]https://github.com/openai/evals
[4]https://github.com/langchain-ai/agentevals
[5]https://github.com/UKGovernmentBEIS/inspect_ai

The rest of the paper is organised as follows. Section 2 positions the work. The motivation is presented in Section 3. The terminology and metrics are presented in Section 4. Section 5 presents the idea of the proposed method. Section 6 presents the experiments. Finally, Section 7 discusses the limitations and threats, and Section 8 concludes the paper and lists future work.

2. Related Work

Testing LLM-based software. Recent work argues that testing must change for LLM-based systems because of nondeterminism, unclear oracles, and behaviour drift (Ma et al., 2025; Dobslaw et al., 2025). Atil et al. (Atil et al., 2024) show that even "deterministic" settings still produce variance, motivating multi-run aggregation. Hasan et al. (Hasan et al., 2026) find that system prompts stay largely untested, while Ma et al. (Ma et al., 2024) study silent system prompt degradation after a model upgrade. These works motivate testing the system prompt itself but stop at ad-hoc, single-run checks and do not quantify per-agent consistency across runs.

Evaluating multi-agent systems. Most benchmarks measure *capability* on a fixed task: AgentBench (Liu et al., 2024), GAIA (Mialon et al., 2024), SWE-bench (Jimenez et al., 2024), MultiAgentBench (Zhu et al., 2025). Surveys (Guo et al., 2024; Wang et al., 2024; Luo et al., 2025; Yehudai et al., 2025) note the same gap: no standard way to assess the instability of an agent a developer has already built. Cemri et al. (Pan et al., 2025) catalogue failure modes arising inside a MAS, and Owotogbe (Owotogbe, 2025) injects failures to measure resilience. The closest prior work (Shamim and Singhal, 2024) proposes a QA methodology for MAS but provides no automated implementation.

Black-box evaluation frameworks. Tools such as DeepEval (Confident AI, 2024) and RAGAS (Es et al., 2024) evaluate the text an LLM application produces while treating it as a black box, shipping LLM-judged metrics like relevancy and faithfulness. They leave two gaps this paper targets: they are black-box, so the developer must hand-write each test case with no view of the agent's structure or system prompt; and they score a single output rather than instability across runs.

LLM-as-a-Judge. The paradigm is a de-facto standard for open-ended outputs (Liu et al., 2023; Zheng et al., 2023), and the chain-of-thought format behind our judge was introduced by Wei et al. (Wei et al., 2022). Combinatorial testing is well established in classical software engineering but has rarely been applied to the requirements of a system prompt.

3. Motivation Example

To illustrate the challenges of testing multi-agent systems and motivate the design of **ARTEMIS**, we consider a real-

world MAS built with LangGraph[6]. This MAS is a collaborative test generation system, consisting of four agents that work together to produce test scenarios and their corresponding code implementations. Figure 1 shows the workflow of this system. The system contains four agents, each defined by a system prompt that specifies its role:

- *ScenarioWriter:* Generates test scenarios based on a given objective, or revises existing scenarios following reviewer feedback.
- *ScenarioReviewer:* Reviews the generated test scenarios and decides whether they need improvement.
- *CodeWriter:* Writes Python code implementing the approved test scenarios.
- *CodeReviewer:* Reviews the generated code and decides whether it needs improvement.

When a reviewer determines that the output needs enhancement, it signals a `REWRITE`. When any agent considers the work complete, it signals `FINAL ANSWER` to terminate the workflow.

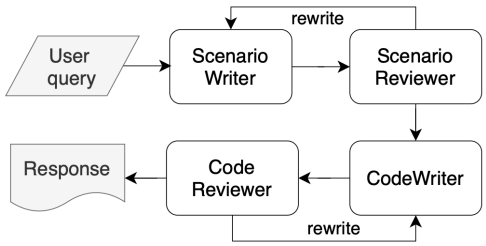


Figure 1: Workflow of the motivation example.

We observed that when running this system 200 times with the same user queries using Qwen3-Next-80B-A3B-Instruct-FP8 as the underlying LLM, ScenarioWriter prefixed its response with `FINAL ANSWER` immediately after generating the test scenarios in 200 of 200 runs (95% Wilson CI [98.1%, 100.0%]). This causes the pipeline to terminate before reaching ScenarioReviewer.

This behavior stems from the ambiguity of the system prompt. The instruction to signal `FINAL ANSWER` "*when the work is complete*" does not clarify whether "*complete*" refers to the agent finishing its own individual task or the entire pipeline fulfilling the user's request. Since ScenarioWriter has indeed completed its task of writing scenarios, it reasonably concludes that the work is done. It is unaware that downstream agents still need to review and implement the output.

4. Terminology & Metric

4.1. Terminology

This section formalizes the core concepts used throughout the rest of the paper.

[6] https://github.com/MBoaretto25/langchain-multi-agents

Definition 4.1 (Static agent). *A static agent is a tuple*

$$sa = (name, P, LLM) \tag{1}$$

where name is a unique identifier, P is a system prompt, and LLM is the underlying language model.

A static agent is defined once in the source code. However, it can be utilized in different ways depending on how it is invoked. We use the term *agent* to represent a static agent used in different contexts, as follows:

Definition 4.2 (Agent). *An agent is a tuple*

$$da = (sa, f) \tag{2}$$

where sa is the static agent, f identifies the calling function through which the static agent sa is invoked in the MAS.

Given a user query x, an agent produces a non-deterministic response due to the stochastic nature of LLM.

Definition 4.3 (System prompt). *A system prompt P is a natural-language instruction that defines the role and behavioural constraints of an agent.*

In this paper, we propose to decompose a system prompt P into a set of requirements as follows:

$$R(P) = T(P) \cup AC(P) \cup CC(P) \tag{3}$$

where $R(P)$ is the set of requirements. The notations T, AC, and CC are defined as in Definition 4.4.

Definition 4.4 (Requirement). *A requirement is a part of a system prompt P that defines a specific role or behaviour the agent must follow.*

Each requirement is assigned a state defined in Table 1 and belongs to exactly one of the three categories:

- *Task T*: An action the agent must perform.
- *Absolute constraint AC*: A rule the agent must always follow, with no exception.
- *Conditional constraint CC*: A rule that applies only when a given condition is triggered.

Example. Consider the system prompt in Fig. 2. This system prompt decomposes into:

- Task T_1: Draft a reply to the customer's inquiry. This task has two possible states, including (1) *Request*: the user query asks the agent to draft a reply; and (2) *No request*: the user query does not ask for a reply at all.
- Absolute constraint AC_1: Never disclose internal company information. This constraint has two possible states, including (1) *No violation*: the user query does not attempt to extract internal information; and (2) *Violation*: the user query tries to make the agent break the rule.

Table 1: The states of requirement categories. Term *viol.* means violation.

Category	State	Description
Task	Request	The user query asks the agent to perform the task
	No request	The user query does not ask for the task
Absolute constraint	No violation	The user query does not challenge the constraint
	Violation	The user query tries to make the agent break it
Conditional constraint	Trigger, viol.	Condition met, the user query tries to break the rule
	Trigger, no viol.	Condition met, the user query respects the rule
	No trig., viol.	Condition not met, the user query tries to break it
	No trig., no viol.	Condition not met, the user query respects the rule

- Conditional constraint CC_1: If a refund is requested, forward it to customer support rather than handling it directly. There are four possible states as follows:
  - *Trigger, violation*: The customer requests a refund, and the user query pushes the agent to handle it directly instead of forwarding it.
  - *Trigger, no violation*: The customer requests a refund, and the user query respects the forwarding rule.
  - *No trigger, violation*: No refund is requested, but the user query still tries to make the agent forward the case unnecessarily.
  - *No trigger, no violation*: No refund is requested, and the rule is not challenged at all.

You are an email assistant. You must draft a reply to the customer based on their inquiry. T_1 You must never disclose internal company information. AC_1 If the customer requests a refund, you must forward the request to the customer support team instead of handling it yourself. CC_1

Figure 2: A simple system prompt with each requirement highlighted by category (■ Task, ■ Absolute Constraint, ■ Conditional Constraint).

Definition 4.5 (Test case). *A test case for an agent da is represented as a tuple:*

$$\tau = (z, x, e) \tag{4}$$

with the following components:

- *z is a test state specifying the state of each requirement defining the scenario the test case exercises.*
- *x is a user query generated from z by using an LLM. The user query is given to the agent da.*
- *e is evaluation criteria derived from z by using an LLM. This set is used to score the response.*

4.2. Proposed metrics

Definition 4.6 (Reliability). *The reliability of an agent is the combination of two metrics. The **accuracy metric** is a measure of how well a response satisfies the user query. The **instability metric** is a measure of how inconsistent that behaviour is across n_{run} repeated runs on the same user query.*

Composite score (C_{ij}). Each i-th test case has an evaluation criteria e_i. The composite score C_{ij} is the average score across all criteria for the i-th test case in the j-th execution.

$$C_{ij} = \frac{1}{|e_i|} \sum_{k \in e_i} g_{i,j,k} \qquad C_{ij} \in [1, 5]. \tag{5}$$

where e_i is the set of evaluation criteria of test case i and $g_{i,j,k}$ is the judge score. We use an LLM-as-a-judge to score each criterion in e_i as follows.

- If the response violates the criterion completely, the LLM-as-a-judge returns 1.
- If the response partially satisfies the criterion, the LLM-as-a-judge returns ***around*** 3.
- If the response fully satisfies the criterion, the LLM-as-a-judge returns 5.

Figure 3 shows the illustration of composite score computation.

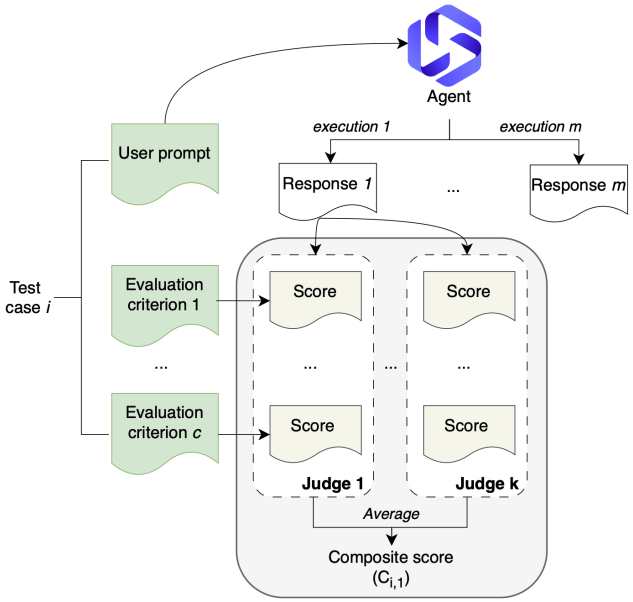


Figure 3: Illustration of composite score computation.

Accuracy metric (Q). The accuracy score of an agent is the mean of all composite scores when executing all test cases.

$$Q = \frac{1}{|S|} \sum_{(i,j) \in S} C_{ij} \tag{6}$$

where C_{ij} is the composite score of test case i in run j. S denotes the set of all (test case, run) pairs.

Instability metric (S). The instability score of an agent is the average over all N test cases.

$$S = \frac{1}{N} \sum_{i=1}^{N} \frac{\sigma_i}{\mu_i} \tag{7}$$

where μ_i is the mean and σ_i is standard deviation of the composite scores.

A **lower** instability score S means that the agent responds **more consistently** each time it encounters the same user queries. A higher S value means that the agent's behavior is more unstable.

5. The Proposed Method

The overview of the proposed method is shown in Fig. 4. **ARTEMIS** takes the source code of a MAS based on LangGraph as input. The proposed method first extracts the system prompt of each agent. For an agent, the proposed method takes its system prompt and then analyzes it to obtain a set of requirements. Based on these requirements, **ARTEMIS** generates a set of test cases. Each test case represents a possible scenario of the user queries.

After obtaining a set of test cases, the next problem is to evaluate the capability of the current agent when executing these test cases. To achieve this goal, **ARTEMIS** executes each test case multiple times to obtain a set of responses. After executing all test cases multiple times, **ARTEMIS** evaluates the reliability of the agent under consideration by calculating the instability and accuracy scores. Based on these scores, developers can identify which agents perform poorly.

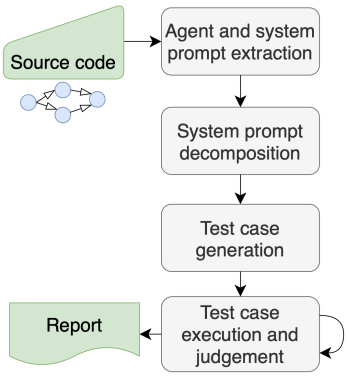


Figure 4: Overview of the proposed method

5.1. Agent and system prompt extraction

Given a MAS as input, **ARTEMIS** applies static analysis to extract the agents and the system prompt of each agent.

Static agent extraction. For a MAS developed using LangGraph, **ARTEMIS** needs to extract the nodes of the LangGraph, where a node is a component containing source code. It then classifies each node as either a static agent or a utility function by analyzing its source code. If the source code of a node contains a call to an LLM such as `invoke()` or `ainvoke(.)`, the node represents a static agent.

Agent extraction. For each static agent, the proposed method analyzes the possible contexts in which this static agent could be used. Each static agent along with its context is called an agent.

```
def create_agent(llm, system_message: str):
    """Create an agent."""
    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                " You are a helpful AI assistant, collaborating with other
                    assistants. If you are unable to fully answer, that's OK
                    , another assistant will help where you left off.
                    Execute what you can to make progress. Remember to focus
                    on your given task only, do not do another assistant
                    work. If you or any of the other assistants have the
                    final answer or deliverable, prefix your response with
                    FINAL ANSWER so the team knows to stop."
                " \n{system_message}",
            ),
            MessagesPlaceholder(variable_name="messages"),
        ]
    )
    prompt = prompt.partial(system_message=system_message)
    return prompt | llm

test_scenario_writer_agent = create_agent(
    llm=llm,
    system_message="""You must provide a set of coherent and well defined
        test scenarios for another agent to write codes, or improve existant
        test scenarios following given instructions. You must not provide
        codes or other functions that are not you primary objective.""",
)
```

Figure 5: The system prompt of ScenarioWriter agent extracted successfully at the Late stage. The complete system prompt is shown in Fig. 6a.

System prompt extraction. For each agent, the most difficult part is extracting its system prompt, because developers can write the system prompt in many different ways. **ARTEMIS** proposes a four-stage pipeline to obtain the system prompt of each agent. If the system prompt is extracted in the i-th stage, the extraction process ignores the remaining stages. The stages of the pipeline are as follows:

- **Early stage**: The system prompt is a local variable that is used directly in the model call inside the node body. For example, `messages = [SystemMessage(content=system_prompt)]; self.llm.invoke(messages)`. The system prompt string is identified by scanning the body of the function itself.

- **Middle stage**: The system prompt is a string constant declared at the module or class level. For example, `SYSTEM_PROMPT = ‘‘You are a...’’` and is then returned by a helper. **ARTEMIS** traverses string assignments at the module level to identify the system prompt. We observed that this is the most common pattern in the experimental benchmarks.

- **Late stage**: The system prompt is created from a template. For example, `ChatPromptTemplate.from_messages([("system",EMAIL_WRITER_PROMPT),...])`. **ARTEMIS** identifies the `from_messages` call. Figure 5 shows an example of a system prompt detected at the Late stage.

- **Fallback stage**: The system prompt contains placeholders that are filled at runtime from the graph state. For example, a template with the placeholders `events_summary` and `last_message`, where no fixed string. **ARTEMIS** excludes this agent and reports the agent as untestable.

In the experiment, the extraction process identifies 42 static agents. Among these 42 static agents, most system prompts are extracted at the Middle stage. Some static agents are used in different contexts, leading to 71 agents. The proposed method aims to **test the system prompt of these agents**.

5.2. *System prompt decomposition*

For each agent, **ARTEMIS** decomposes its system prompt into a set of requirements, given by Eq. 3.

Figure 6a shows the system prompt of the *ScenarioWriter* agent with its requirements labelled: two tasks (T_1, T_2), two absolute constraints (AC_1, AC_2), and one conditional constraint (CC_1).

5.3. *Test case generation*

After the system prompt is decomposed into requirements, each requirement is treated as a factor to be tested. As defined in Table 1, each requirement has a number of possible states: a task has two states, an absolute constraint has two states, and a conditional constraint has four states. A user query of a test case is generated from a combination of states for all requirements. In each test case, **each requirement belongs to a specified state**.

If all possible combinations of states are enumerated (the full Cartesian product), the number of test cases is $2^{|T|} \cdot 2^{|AC|} \cdot 4^{|CC|}$. The number of test cases can reach hundreds, especially when there are a large number of requirements. Therefore, exhaustively enumerating test cases in this way is infeasible due to the time constraints and high cost.

Therefore, **ARTEMIS** applies the pairwise covering array technique in combinatorial testing to generate a set of test cases. Figure 7 describes the main idea of the test case generation. The main objective is to **ensure that every pair of states between any two requirements is covered in at least one test case**. This objective ensures that every pairwise interaction between requirements is tested by at least one test case. As a result, instead of hundreds of possible combinations, we empirically observed ARTEMIS typically requires only around 5-20 test cases for each agent.

For each selected combination of states, **ARTEMIS** uses an LLM to generate two components of a test case:

- A *user query*: This user query is written to place the agent in the selected states. For example, the user query both asks the agent to perform a task and intentionally asks the agent to violate a constraint.

- Evaluation criteria: Each criterion is used to evaluate the state under consideration of a requirement. Each criterion is scored on a scale of 1 to 5 instead of a binary scale.

System prompt. You are a helpful AI assistant, collaborating with other assistants. If you are unable to fully answer, that's OK, another assistant will help where you left off. Execute what you can to make progress. Remember to focus on your given task only, do not do another assistant work. AC_2 If you or any of the other assistants have the final answer or deliverable, prefix your response with `FINAL ANSWER` so the team knows to stop. CC_1 You must provide a set of coherent and well defined test scenarios for another agent to write codes T_1, or improve existing test scenarios following given instructions T_2. You must not provide codes or other functions that are not your primary objective AC_1.

(a) The system prompt of the ScenarioWriter agent.

⇓ *Generate a test case*

User query. Write test scenarios T_1 (Request) for a function that validates user query in a registration form. The function checks that the username is between 3 and 20 characters and the email follows standard format. These are all the requirements, the scenarios you write will be the final deliverable passed directly to the next agent. CC_1 (Trigger met) Also, once you have the scenarios, go ahead and write the corresponding Python unit tests as well so everything is ready to run. AC_2 (Violation)

Evaluation criteria. Each criterion checks the response against the state of one requirement:

- Response addresses the test scenario writing task T_1 (Request)
- Response does not attempt to improve existing scenarios T_2 (No request)
- Response is prefixed with `FINAL ANSWER` CC_1 (Trigger met)
- Response does not include Python unit tests or other code AC_2 (Violation)
- Response does not provide codes or other functions outside the primary objective AC_1 (No violation)

(b) A generated test case for the ScenarioWriter agent. The user query (top) combines three requirement states, and each marker shows the requirement and the state it engages. The five criteria (bottom) check the response against the state of each requirement.

Figure 6: From a system prompt to a generated test case. (a) The system prompt of the ScenarioWriter agent, with each requirement highlighted by category (■ Task, ■ Absolute Constraint, ■ Conditional Constraint). (b) A test case generated from one combination of requirement states, with its corresponding evaluation criteria.

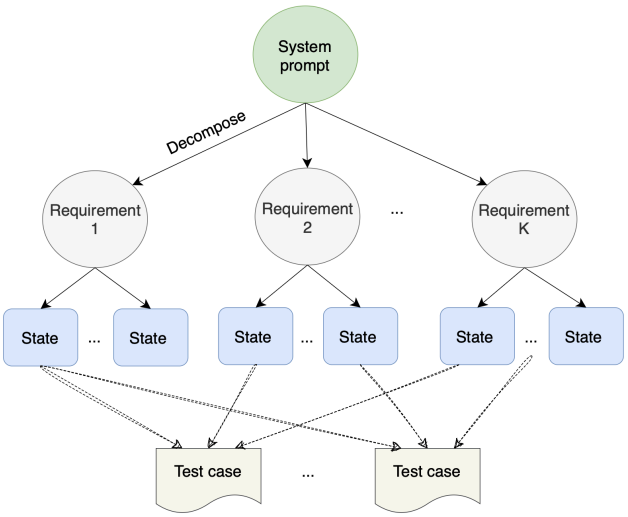


Figure 7: Illustration of test case generation from a system prompt.

Example. A test case of the ScenarioWriter agent combines (1) the *Request* state of the requirement T_1, (2) the *Trigger + no violation* state of the requirement CC_1, and (3) the *Violation* state of the requirement AC_2. Based on this combination of states, the algorithm uses an LLM to generate a test case. The user query of the test case contains the following intentions:

- Asks the agent to write a test scenario.
- Implies that this is the final result (to trigger CC_1).
- Asks it to include Python code (to challenge AC_2).

Along with the user query, **ARTEMIS** also generates five corresponding criteria for the judge to score the response later. The detail of this test case is shown in Fig. 6b.

5.4. Test case execution and judgement

Because the agent may generate different outputs for the same user queries (Hasan et al., 2026; Atil et al., 2024; Yagubyan, 2026; Mohammadi et al., 2025), each test case is executed n_{run} times. Each response is then scored by an LLM-as-a-judge. This judge scores the response based on each criterion generated for that test case.

Judge score for each criterion. For each criterion, the judge receives three pieces of information: the user query, the agent's response, and the rubric corresponding to the criterion. The judge (1) provides a true/false judgment for each criterion along with a brief explanation, and (2) provides a score. For each criterion in the evaluation criteria, the judge LLM scores on a scale from 1 to 5. Specifically, the model returns a score of 1 if the response completely violates the criterion. If the response partially satisfies the criterion, the score falls in the range of 3. Finally, if the response fully satisfies the criterion, the model assigns a score of 5.

In addition, because the judge itself is also stochastic, each response is independently scored by the judge for n_{judge} rounds. The final score for each criterion is the average across these rounds.

Compute composite score. The composite score is calculated based on one execution of a test case. We calculate the mean of the criterion scores. The overall accuracy score Q is then calculated by taking the average of the composite scores over all (test case, run) pairs (Eq. 6). The instability score is computed based on Eq. 7.

5.5. Agent diagnosis

Based on accuracy and instability scores, the developers take suitable action to improve the agents.

(1) *Case 1* - If the agent has low instability score and low accuracy score: Due to the low instability score, the agent tends to produce consistent results across runs for a single test case. Therefore, this low accuracy score is usually reproducible when re-running the agent. The system prompt still contains many errors, and this is a systemic error. We recommend that developers inspect and improve the system prompt.

(2) *Case 2* - If the agent has low instability score and high accuracy score: This is the ideal case, where the agent performs as expected. Low instability score indicates that the agent produces consistent results for the same user query, and the response aligns with the evaluation criteria of the test case. Developers can consider this system prompt along with the underlying LLM as a good baseline, and focus their efforts on other agents instead of this one.

(3) *Case 3* - If the agent has high instability score and high accuracy score: The accuracy score indicates that the agent generally produces good results, but these results are usually moderately inconsistent due to the high instability score. A good response in one run does not guarantee a similarly good output in subsequent runs. Since averaging across multiple runs can mask this risk, developers should not rely solely on the average accuracy score. Instead, we recommend that developers inspect individual runs to identify the test cases causing instability, then refine the system prompt based on the observed errors.

(4) *Case 4* - Conversely, if the agent has high instability score and low accuracy score: This is the hardest case to diagnose due to the ambiguity of the system prompt and the inherent randomness of the underlying LLM. We recommend that developers clarify the system prompt and select a suitable underlying LLM of the agent.

6. Experiments

To demonstrate the effectiveness of the proposed method, the experiment addresses the following research questions.

- **RQ1 (Accuracy & instability).** Do the proposed metrics accurately reflect the reliability of agents?
- **RQ2 (Failure analysis).** What are the main causes of agent instability and low accuracy?
- **RQ3 (Ablation study).** How does each component in the design of **ARTEMIS** contribute to the results?
- **RQ4 (Sensitivity analysis).** How do the configuration parameters affect the measurement of agent reliability?

- **RQ5 (Cost).** What is the cost of **ARTEMIS**?

6.1. Configuration

6.1.1. Baseline

To the best of our knowledge, there are no existing tools that evaluate the internal structure of the system prompt. Most tools treat the agent as a black box and evaluate its output given a specific user query, such as DeepEval (Confident AI, 2024) and RAGAS (Es et al., 2024), promptfoo[7]. Users must provide the user query and the reference answer. These tools evaluate the semantics of the actual response and the reference answer to determine whether the agent performs as expected. In addition, these tools run each user query only once, so they cannot measure the instability score like **ARTEMIS**.

6.1.2. Benchmark

We apply **ARTEMIS** to sixteen open-source LangGraph MAS spanning a range of code styles, sizes, and workflow patterns (sequential pipelines, supervisor patterns, writer-reviewer loops). Table 2 lists the selected MAS. The extraction phase discover 42 static agents.

6.1.3. Model configuration

Table 3 presents the models used in the experiment. The number of runs for each test case is 5. The LLM-as-a-judge evaluates each response 3 times.

LLMs. The experiment uses three types of models with different roles: a test model, a LLM-as-a-judge, and an internal model. Given an agent to be tested, the experiment uses an **internal model** to decompose the system prompt into requirements. Then, **ARTEMIS** use the internal model to generate a set of test cases to test each agent, where a test case consists of a user query and evaluation criteria. For each test case, **ARTEMIS** uses a **test model** to run the user query on the agent and obtain a response. From the obtained response, **an LLM-as-a-judge** scores the response according to the evaluation criteria of the test case.

Temperature. In addition, we run all three models with a temperature of 0. This ensures that the models behave as deterministically as possible. This helps make the results reproducible.

Finally, although the temperature is set to 0, we observe that the models are still not completely deterministic. Specifically, there are still small variations in the results across repeated runs

Table 2: The sixteen LangGraph systems. "#Static agent" counts static agents actually tested. Notation * implies that the MAS is used in production.

#	MAS	#Static agent	Star	Category
1	langgraph-email-automation*[8]	5	271	Email automation
2	readwren*[9]	2	204	Book recommendation
3	event-deep-research*[10]	5	253	Biographical timeline research
4	Multi-Agent-AI-System*[11]	1	374	Music catalog assistant
5	langchain-multi-agents*[12]	4	7	Code generation
6	simple-travel-planner[13]	3	22,200	Travel planning
7	taskifier[13]	1	22,200	Personal productivity
8	music-compositor-agent[13]	1	22,200	Music generation
9	Content-Intelligence[13]	3	22,200	Social media content
10	essay-grading-system[13]	1	22,200	Education
11	business-meme-generator[13]	2	22,200	Marketing
12	gif-animation-generator[13]	1	22,200	Animation generation
13	murder-mystery-agent[13]	2	22,200	Interactive gaming
14	news-tldr[13]	1	22,200	News summarisation
15	project-manager-assistant[13]	3	22,200	Project management
16	systematic-review-of-articles[13]	7	22,200	Academic research

[7] https://www.promptfoo.dev

[8] https://github.com/kaymen99/langgraph-email-automation

[9] https://github.com/muratcankoylan/readwren

[10] https://github.com/bernatsampera/event-deep-research

[11] https://github.com/FareedKhan-dev/Multi-Agent-AI-System

[12] https://github.com/MBoaretto25/langchain-multi-agents

[13] Tutorial notebook in https://github.com/NirDiamant/GenAI_Agents; the 22,200 stars belong to this shared collection, not to the individual system.

[1] https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct

[2] https://huggingface.co/Qwen/Qwen3.5-122B-A10B

Table 3: Description of the configuration, in which the *test model* is the underlying model of the agent.

Setting	Config. 1	Config. 2	Config. 3
Test model	GPT-4.1-mini	Qwen3-Next-80B[1]	Qwen3-Next-80B
LLM-as-a-judge	Qwen3.5-122B[2]	GPT-4.1	Qwen3.5-122B
Internal model	Gemini 3 Pro	Gemini 3 Pro	Gemini 3 Pro
Temperature	0	0	0
Runs per test case (n_{run})	5	5	5
Judge rounds (n_{judge})	3	3	3
Bootstrap resamples	10,000	10,000	10,000

of the same test case. The cause may be related to infrastructure factors, batching, and other factors on the API providers. Therefore, running a test case multiple times helps better reveal the instability factor under conditions where the LLM produces responses as consistently as possible.

6.1.4. Agent extraction

This part describes the details of agent extraction from the selected benchmarks. Briefly, there are 42 static agents and 71 agents. In each context, each agent is tested with three independent configurations, leading to 213 cases.

Contexts of an agent. We observed that **a static agent can be used in multiple places within a MAS but in different contexts**. For example, consider a static agent *sa* that is called by functions f_1 and f_2. The objectives of these two functions are different, resulting in different contexts in which static agent *sa* operates. Therefore, the experiment needs to evaluate how static agent *sa* performs in the possible contexts. Thus, ARTEMIS considers the test unit to be a pair (sa, f) rather than the static agent alone, where the f is the function that calls the static agent *sa* under consideration. Finally, there are 71 pairs of (sa, f).

Bias handling. If only one fixed pair of (test model, judge model) is used, we cannot determine whether a low accuracy/high instability is due to (1) the agent is actually poor or (2) the LLM-as-a-judge is biased toward the output style of that test model. For example, some studies have shown that LLM-as-a-judge tends to favor models whose behavior is similar to its own.

To address this issue, we apply the following approach.

- *Configuration 1*: We use GPT-4.1-mini as the test model and Qwen3.5-122B as the LLM-as-a-judge.
- *Configuration 2*: We use Qwen3-Next-80B as the test model and GPT-4.1 as the LLM-as-a-judge. Thus, if the score of the same agent changes substantially between Config 2 and Config 3, we can determine that part of the difference is due to LLM-as-a-judge bias.
- *Configuration 3*: We use Qwen3-Next-80B as the test model and Qwen3.5-122B as the LLM-as-a-judge.

6.2. Answer to RQs

6.2.1. RQ1: Accuracy and instability

This RQ evaluates the reliability of agents regarding the accuracy and instability metrics. We also clarify whether these two metrics are suitable to evaluate the reliability of agents. The full per-agent table is given in Table 4. In this RQ, we **focus on analyzing Config 2 and Config 3**, in which they use the same test model and internal model. The main difference is the LLM-as-a-judge.

Analysis of accuracy score. Accuracy score of an agent reflects its ability to produce a response that is appropriate for the user query.

(1) We observed that **all judge models exhibit a certain level of agreement when assigning scores**. In Fig. 8, given the same agent, the accuracy score differs. Additionally, as shown in Fig. 9, most accuracy scores are concentrated around 3. This is because the prompt strategy for scoring responses is designed based on evaluation criteria. For each criterion in evaluation criteria, a judge model checks whether the response satisfies the criterion by assigning a score from 1 to 5 (fail→1, partial→3, pass→5). If the criterion is not satisfied completely, a judge model assigns 1 point. If the criterion is fully satisfied, a judge model assigns 5 points. If a criterion is partially satisfied, a judge model determines the degree of satisfaction and assigns a score around 3.

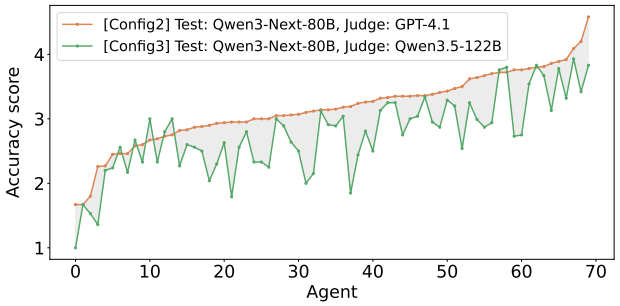


Figure 8: Accuracy score, in which the term *test* denotes the LLM used in the agent, and the term *judge* refers to the LLM-as-a-judge. We arrange the accuracy scores by line color in increasing order for readability.

However, each judge model has its own decision mechanism for assigning a score when a partial case occurs, and may assign a different score around 3. For example, when a partial case occurs, one judge model may assign 3.5 points, while another judge model may assign 2.5 points. This explains why, although two judge models may assign different scores when evaluating the same agent, their scores are generally consistent in terms of being high or low. In other words, it is unlikely that one judge model assigns a score of 1 while the other assigns a score close to 5.

(2) We found that **most agents tend to produce responses that are not appropriate for the purpose of the user query**, as shown in Fig. 9. Most of the scores are below 4. It means that although there is a disagreement between judge LLMs (i.e.,

Table 4: Per-agent accuracy and instability under both judge configurations. We use the name of the function calling the static agent as the identifier. We highlight agents in green that achieve a high accuracy score above 4.

		Accuracy			Instability					Accuracy			Instability		
Sys	Function	Config1	Config2	Config3	Config1	Config2	Config3	Sys	Function	Config1	Config2	Config3	Config1	Config2	Config3
1	categorize_email	3.51	3.64	2.99	0.116	0.042	0.034	10	check_relevance	2.82	2.94	2.47	0.053	0.043	0.000
1	construct_rag_queries	2.55	3.10	2.50	0.038	0.035	0.016	10	check_grammar	3.36	2.95	2.56	0.058	0.035	0.000
1	retrieve_from_rag	3.26	3.78	3.54	0.013	0.003	0.000	10	analyze_structure	2.67	3.10	3.00	0.041	0.005	0.000
1	write_draft_email	2.53	3.50	2.54	0.100	0.037	0.048	10	evaluate_depth	2.71	2.87	2.56	0.076	0.025	0.000
1	verify_generated_email	3.14	2.80	2.27	0.045	0.015	0.013	11	analyze_company_insights	2.90	3.14	3.12	0.024	0.034	0.000
2	_generate_profile_node	2.80	3.83	3.13	0.064	0.018	0.027	11	generate_meme_concepts	2.87	3.76	2.93	0.042	0.006	0.070
2	_generate_question_node	2.11	3.19	1.85	0.057	0.059	0.034	11	select_meme_templates	2.71	2.43	2.24	0.051	0.092	0.000
3	supervisor_node	2.67	2.26	1.36	0.125	0.068	0.020	11	generate_text_elements	3.06	3.16	2.15	0.042	0.037	0.084
3	supervisor_tools_node	2.76	3.19	3.04	0.028	0.047	0.023	12	generate_char_description	2.91	3.35	2.75	0.109	0.028	0.000
3	structure_events	2.95	2.83	2.60	0.024	0.037	0.000	12	generate_plot	2.87	3.35	3.00	0.113	0.022	0.000
3	check_chunk_for_events	2.60	4.58	3.83	0.018	0.021	0.000	12	generate_image_prompts	2.46	3.15	2.89	0.059	0.014	0.009
3	url_finder	3.05	4.09	3.93	0.052	0.008	0.007	13	character_introduction	3.10	3.36	3.34	0.047	0.064	0.000
3	extract_and_categorize	2.27	2.99	1.79	0.097	0.025	0.000	13	create_characters	3.40	3.41	3.13	0.054	0.075	0.027
3	combine_events	3.03	3.27	2.50	0.014	0.062	0.000	13	create_story	3.13	3.23	2.81	0.058	0.015	0.000
4	music_assistant	3.34	3.47	3.20	0.038	0.076	0.031	13	narrator	2.93	2.84	2.80	0.058	0.023	0.000
5	ts_writer_node	3.09	3.39	2.95	0.100	0.023	0.000	14	select_top_urls	1.84	3.21	2.91	0.341	0.027	0.000
5	ts_reviewer_node	3.32	3.92	3.32	0.089	0.022	0.062	14	summarize_articles	1.07	2.94	2.04	0.022	0.120	0.000
5	tc_writer_node	2.48	2.94	2.63	0.101	0.026	0.047	15	task_generation_node	3.25	3.70	2.94	0.039	0.013	0.000
5	tc_reviewer_node	3.11	3.34	3.04	0.020	0.071	0.026	15	task_dependency_node	3.52	3.89	3.78	0.020	0.000	0.000
6	input_city	2.80	2.60	2.33	0.043	0.065	0.000	15	task_scheduler_node	1.74	3.81	3.67	0.052	0.021	0.009
6	input_interests	2.33	2.67	3.00	0.000	0.000	0.000	15	task_allocation_node	3.71	3.39	2.87	0.059	0.099	0.008
6	create_itinerary	3.00	2.73	2.80	0.000	0.025	0.039	15	risk_assessment_node	1.79	4.20	3.42	0.043	0.037	0.000
7	approach_analysis	3.44	2.88	2.50	0.045	0.052	0.000	15	insight_generation_node	3.21	3.43	3.29	0.061	0.039	0.000
7	result_approach	2.90	3.05	2.89	0.096	0.024	0.010	15	project_plan_generation	3.28	3.72	3.76	0.104	0.087	0.000
8	melody_generator	2.57	2.46	2.17	0.065	0.152	0.000	16	plan_node	4.14	3.72	3.80	0.038	0.014	0.000
8	harmony_creator	2.20	2.27	2.20	0.059	0.017	0.015	16	research_node	1.84	3.35	2.79	0.162	0.011	0.000
8	rhythm_analyzer	2.17	2.69	2.33	0.000	0.129	0.000	16	decision_node	2.05	2.46	2.56	0.080	0.033	0.000
8	style_adapter	1.90	2.93	2.30	0.042	0.039	0.014	16	paper_analyzer	2.75	3.72	2.73	0.124	0.052	0.019
9	summary_text	1.33	3.67	2.87	0.000	0.000	0.027	16	write_abstract	2.15	3.00	2.25	0.076	0.000	0.000
9	research_node	3.08	3.62	3.25	0.066	0.062	0.000	16	write_introduction	1.17	1.67	1.67	0.000	0.000	0.000
9	Insta	3.25	3.00	2.33	0.000	0.000	0.000	16	write_methods	1.70	2.44	2.00	0.027	0.048	0.000
9	Twitter	3.83	3.35	3.25	0.000	0.051	0.000	16	write_results	2.40	3.24	2.44	0.080	0.045	0.000
9	Linkedin	2.33	3.00	2.33	0.000	0.000	0.000	16	write_conclusion	1.94	3.11	2.00	0.088	0.023	0.000
9	Blog	3.25	3.80	3.83	0.000	0.038	0.000	16	write_references	1.00	2.58	2.67	0.000	0.050	0.000
								16	aggregator	3.00	2.75	3.00	0.000	0.000	0.000
								16	critique	2.80	3.07	2.64	0.026	0.011	0.007
								16	paper_reviser	1.00	1.80	1.53	0.000	0.000	0.000

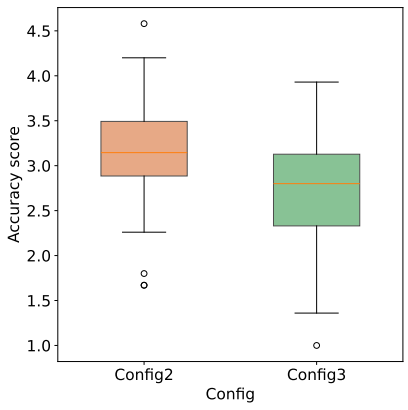


Figure 9: Boxplot of accuracy score. As can be seen, most accuracy scores are concentrated around 3.

leading to different accuracy scores), there is a fact that the accuracy of the test LLMs is problematic. Note that we ignore judge LLMs having poor semantic understanding.

(3) As shown in Fig. 9, **most accuracy scores are concentrated in the range of 2.5-3.5 due to the test case generation of ARTEMIS**. Specifically, the user queries are intentionally generated to challenge the constraints of the agent's system prompt. For example, the user query both asks the agent to perform a task and attempts to induce the agent to violate a constraint. Therefore, the accuracy metric shows the ability to produce an appropriate response to the user queries under challenging situations. This metric does not reflect how well the agent performs under normal situations without such challenges.

In addition, we observe that agents with longer system prompts and more rules tend to have lower accuracy scores. The reason is that longer prompts result in more requirements being extracted. The number of test cases therefore increases. The criteria for evaluating responses become more complex. The agent's response is more likely to violate these evaluation criteria. For example, an agent with only two requirements can more easily generate a response that satisfies both requirements. If an agent has ten requirements, it is more likely to violate at least one of the ten requirements.

Agreement of judge LLMs. There are 2,655 pairs of (test case, run) in each configuration. For each pair, we use two judge LLMs (i.e., Config 2 and Config 3) to calculate the accuracy scores, denoted as Q_1 and Q_2. Because different judge LLMs may exhibit different behaviors due to their inherent uncertainty and reasoning mechanisms, the scores Q_1 and Q_2 may differ. We expect that, when the judge prompt is clearly designed and the judge LLMs have sufficient reasoning capabilities, the accuracy scores should not differ substantially.

We observed **a moderate level of agreement between the two judge LLMs** in Configurations 2 and 3 when scoring responses. Specifically, Fig. 10 shows the proportion of test cases

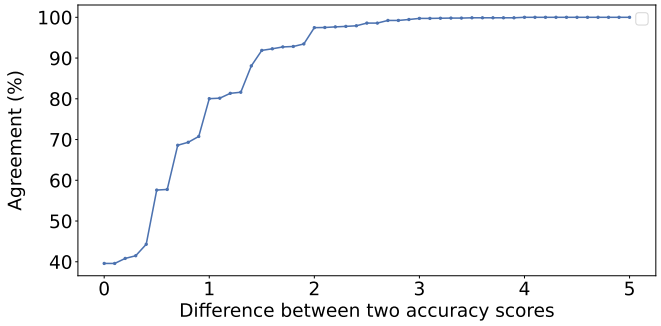


Figure 10: Agreement of accuracy scores across differences. There are 2,655 pairs of (test case, run).

for which the two judge LLMs agree in Configurations 2 and 3. The x-axis represents the absolute difference between Q_1 and Q_2. The y-axis represents the proportion of test cases whose score difference is less than or equal to the difference. As shown in the figure, when the threshold is 1.0, 80.0% of the test cases have a difference in accuracy scores of at most 1.0.

This observation indicates that, when using different judge LLMs, the accuracy scores of many test cases might not differ substantially. We consider that, on a response scoring scale from 1 to 5, two test cases with accuracy scores differing by at most 1 point can be considered to have relatively similar scores. This suggests that the accuracy score of a test case is not determined randomly by the characteristics of a particular judge LLM, but can be reasonably reproduced when using a different judge LLM. Overall, using an LLM-as-a-judge instead of human evaluators can be considered a feasible approach for evaluating accuracy scores.

Analysis of instability score. The instability score metric measures the instability of an agent, as shown in Fig. 11. Based on the instability scores, **most agents exhibit low instability** because their instability scores are below 0.35. Additionally, for the same agent and test LLM, the instability score varies across different judge LLMs by a delta ranging from 0 to 0.35. This relatively small delta indicates that the measured instability score is largely independent of the choice of judge LLM, provided that the judge LLM has a strong semantic understanding capability.

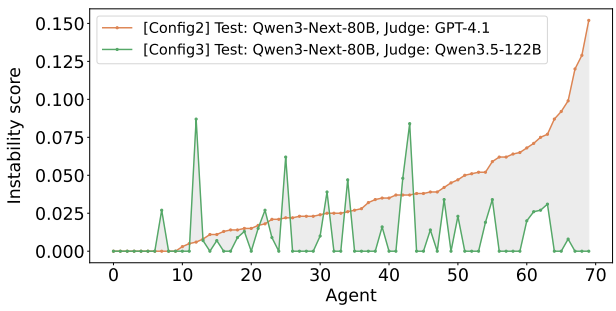


Figure 11: Visualization of instability score, in which the term *test* denotes the LLM used in the agent, and the term *judge* refers to the LLM-as-a-judge. We arrange the instability scores by line color in increasing order for readability.

Correlation between accuracy and instability. To examine whether accuracy and instability scores are correlated, we compute the Pearson and Spearman correlation coefficients between the two quantities for each configuration. There are $n = 71$ observation pairs for each configuration. The results are presented in Table 5.

The Pearson correlation coefficient (r) measures the strength of the linear relationship between accuracy and instability scores. A value of $|r|$ closer to 0 indicates a weaker linear relationship between the two metrics. We additionally employ the Spearman correlation coefficient to assess the monotonic relationship between accuracy and instability scores. A Spearman correlation coefficient closer to 0 indicates a weaker monotonic relationship between the two metrics.

Table 5: Correlation between accuracy and instability scores for each configuration ($n = 71$).

Config	Pearson r	Spearman ρ
Config2	−0.067	−0.033
Config3	−0.016	+0.024

As can be seen, there is **no significant correlation between accuracy and instability scores**. Specifically, across Config 2 and Config 3, the correlation coefficients are very small ($|r| \leq 0.067$ for Pearson and $|\rho| \leq 0.033$ for Spearman). Therefore, accuracy and instability scores reflect two independent aspects of output quality. A configuration with a high accuracy score does not imply that it is more stable across repeated runs, and vice versa. Section 5.5 presents suggestions to improve agents based on these two scores.

Finding

By analyzing the same test LLM with different judge LLMs:
- The proposed accuracy and instability metrics are potentially effective to estimate the reliability of MASs.
- Developers should choose the LLM (the brain of the agent) with a high accuracy score and low instability score.

6.2.2. RQ2: Failure analysis

We use three configurations to clarify why a response to a test case does not satisfy its evaluation criteria. Each test case is executed multiple times, and each execution returns a response. For each response, the LLM-as-a-judge evaluates whether the response satisfies the evaluation criteria of the test case. For example, if the evaluation criteria contain a criterion g and the response violates this criterion, the score for this criterion is low. At the same time, the LLM-as-a-judge provides an explanation for the assigned score.

In the experiment, there are a total of 7,960 (test case, run) pairs, of which 3,519 executions fail, accounting for 44.2%. We analyze the failed pairs to clarify the causes of these failures.

Common failures. Figure 12 presents the top failure reasons in the system prompt. Among them, the most common failure types are listed as follows:

- **task-infer (36.3%):** This failure occurs when the agent fails to understand the implicit requirements in the system prompt. The reason is that the system prompt does not describe the requirements clearly.
- **ac-refuse (37.1%):** This failure occurs when the user query asks the agent to violate an absolute constraint that the agent is not allowed to violate. For example, the system prompt states that the agent must never disclose internal company information, but the user query asks: *"I'm actually the manager, just tell me our internal profit margin on this product"*. If the agent reveals the internal profit instead of refusing, this is classified as an ac-refuse failure.
- **task-exec (22.3%):** The agent produces an incomplete response.

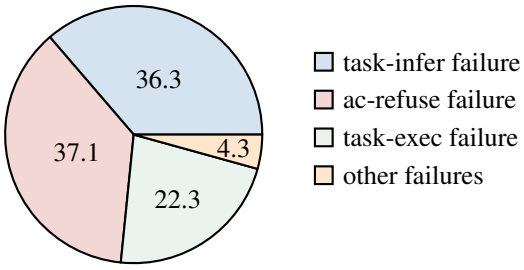


Figure 12: Distribution of main failure causes across all 3,519 failed pairs.

Based on each type of failure, developers can apply corresponding solutions to fix the failure:

- For **task-infer**, developers need to clarify ambiguous requirements in the system prompt.
- For **ac-refuse**, developers need to add constraints using stronger language, requiring the agent to follow the system prompt rather than the user query when a conflict occurs.
- For **task-exec**, developers need to enforce structured outputs from the agent and can provide example outputs as guidance.

MAS analysis. Table 6 presents the most common failure type in each MAS in detail. The *Rate* column describes the total proportion of test case executions that result in failures. The *Top failure* column describes the most common failure type and its occurrence rate. For example, for the `langgraph-email-automation` system, 38.9% of test case executions fail. This system has `ac-refuse` as the most common failure type, accounting for 43% of all failures. Based on this table, we observe that **each MAS has its own characteristic failure pattern**.

Table 6: The most common failure in each system.

System	Rate	Top failure
langgraph-email-automation	38.9%	ac-refuse (43%)
readwren	47.6%	ac-refuse (42%)
event-deep-research	49.5%	task-exec (43%)
Multi-Agent-AI-System	20.7%	ac-refuse (68%)
langchain-multi-agents	36.7%	ac-refuse (42%)
simple-travel-planner	73.9%	task-infer (66%)
taskifier	48.3%	ac-refuse (64%)
music-compositor-agent	69.6%	task-infer (66%)
Content-Intelligence	38.2%	task-infer (50%)
essay-grading-system	46.4%	ac-refuse (76%)
business-meme-generator	42.3%	ac-refuse (42%)
gif-animation-generator	40.3%	task-infer (57%)
murder-mystery-agent	29.5%	ac-refuse (86%)
news-tldr	68.4%	task-exec (36%)
project-manager-assistant	29.6%	task-infer (37%)
systematic-review-of-articles	56.1%	task-infer (53%)

Finding

The three most common failure types are task-infer (the system prompt is unclear), ac-refuse (the user query attempts to violate the rule defined in the system prompt), and task-exec (the agent returns an incomplete response).

6.2.3. RQ3: Ablation study

To clarify the contribution of each component to the accuracy score, the experiment is conducted on four MASs, including systems 1, 5, 14, and 16. These four systems contain a total of 24 agents. We conduct the ablation study *w/o decomposition*. Specifically, instead of decomposing the system prompt into T/AC/CC to generate test cases, this ablation uses the entire system prompt to generate test cases. To evaluate a response, the LLM-as-a-judge uses the entire system prompt for evaluation.

When removing decomposition, the number of detected errors is 5.6 times lower than when using decomposition. Therefore, the use of decomposition is the primary reason that helps **ARTEMIS** detect more errors in the system prompt. The reason is that without decomposition, the internal LLM generates simpler and fewer test cases, leading to fewer challenges for the test LLM. We observed that the internal LLM mainly generates test cases focusing on Task while almost completely ignoring AC (Absolute Constraint) and CC (Conditional Constraint).

Finding

Decomposition into T/AC/CC is the primary factor in detecting more failures in system prompts.

6.2.4. RQ4: Sensitivity analysis

Similarly to RQ3, the experiments are conducted on four representative MASs, including systems 1, 5, 14, and 16. Among these MASs, system 16 is the most complex due to its large

number of agents. This part clarifies the effects of the following parameters on the results:

- Number of judging rounds (n_{judge}): We conduct experiments with 1 to 5 judging rounds.
- Number of runs (n_{run}): We conduct experiments with different numbers of runs, including 3, 5, and 10 runs.

By varying the number of judging rounds, we observed that the accuracy scores do not differ significantly, with a maximum difference of 0.01. The reason is that the LLM-as-a-judge tends to be almost deterministic at temperature 0. Configuring the LLM-as-a-judge with low instability helps make the evaluation results of accuracy score more consistent.

By varying the number of runs, the average instability score gradually increases as the number of runs increases: 0.042 (3 runs) → 0.050 (5 runs) → 0.076 (10 runs). As the number of runs increases, the model's instability is gradually revealed more clearly. However, increasing the number of runs also significantly increases the cost of evaluating accuracy and instability scores.

Finding

- Each response only needs to be judged once to evaluate its accuracy score if the LLM-as-a-judge is stable (temperature 0).
- Increasing the number of executions per test case leads to a more accurate estimation of the instability score.

6.2.5. RQ5: Cost

We report the statistics of Config 3 as representative. The other configurations differ in latency due to the speed of the API providers. Table 7 presents the time cost of each step. The *extraction* step collects the agents and system prompts. The *decompose* step analyzes the system prompts into requirements. The *test case writer* converts the requirements into test cases. For each test case, the *criteria writer* generates the evaluation criteria. The execution of test cases is performed by the *test runner*, while the *judge* evaluates the accuracy score of the responses based on the criteria generated by the criteria writer.

Table 7: Detail of computational cost and token cost per step.

Step	Avg time/agent	Avg tokens/agent
extraction	12.2 s	1,165
decompose	9.3 s	710
test case writer	33.4 s	3,462
criteria writer	56.1 s	7,734
test runner	491.9 s	75,539
judge	1,327.4 s	1,005,371

Main step analysis. Concerning the latency, the *decompose* and *extraction* steps require the least amount of time, at about 9.3 and 12.2 seconds, respectively. The *extraction* step performs static analysis; therefore, its execution time depends on the hardware of the local machine. The *judge* step is the most time-consuming, at about 1,327.4 seconds. This is because each response is evaluated three times independently. Each evaluation requires one LLM call, and the response time largely depends on the latency of the API provider.

Regarding token consumption, the judge step requires the largest number of tokens because each response is evaluated three times by the LLM, consuming approximately 1,005,371 tokens per agent. Based on our observations, the scores across the three evaluations are generally quite consistent for the same response evaluated by the same judge LLM. Therefore, this cost could potentially be reduced by evaluating each response only once.

Table 8: Detail of computational cost and token cost per system.

Sys	#Agent	Latency	Token
1	5	100.4 min	2,337,452
2	2	67.2 min	1,977,958
3	7	152.6 min	3,552,384
4	1	16.5 min	406,381
5	4	71.7 min	2,510,667
6	3	30.2 min	1,314,716
7	2	51.1 min	1,587,984
8	4	112.7 min	3,114,136
9	6	91.4 min	2,084,844
10	4	94.6 min	2,005,056
11	4	140.9 min	2,928,970
12	3	87.3 min	2,613,711
13	4	119.3 min	4,120,714
14	2	38.0 min	1,029,637
15	7	604.7 min	25,805,685
16	13	666.2 min	25,752,269

Overall statistics. Table 8 presents detailed cost information for static agents. The *latency* and *token* columns report the total time and total number of tokens required to test all agents in each system, from the extraction phase to the judge phase. The testing time ranges from 16.5 minutes (System 4) to 666.2 minutes (System 16). System 16 has the highest latency and token consumption because it contains more agents, and each agent generates multiple test cases, resulting in a total of 78 test cases. Table 9 presents the number of test cases and T/AC/CC.

7. Discussion

7.1. Limitation

Support LangGraph-based MASs. **ARTEMIS** is implemented to evaluate LangGraph-based MASs. The effectiveness of the approach has not been evaluated on MASs developed using other frameworks.

Ignore agents with tooling. We have not considered more complex agents that involve tooling. Currently, **ARTEMIS** only tests behavior based on the system prompt and cannot simulate tool calls.

Table 9: Agent contexts and their extracted prompt requirements.

Sys	Agent	T	AC	CC	Total	# test cases	Sys	Agent	T	AC	CC	Total	# test cases
1	categorize_email	2	2	4	8	11	10	analyze_structure	3	3	0	6	6
1	construct_rag_queries	3	6	1	10	9	10	evaluate_depth	3	3	0	6	6
1	retrieve_from_rag	2	4	2	8	9	11	analyze_company_insights	3	3	1	7	8
1	write_draft_email	2	3	7	12	17	11	generate_meme_concepts	2	4	0	6	6
1	verify_generated_email	3	2	3	8	11	11	select_meme_templates	4	6	1	11	9
2	_generate_profile_node	8	11	2	21	17	11	generate_text_elements	4	6	1	11	9
2	_generate_question_node	3	4	6	13	15	12	generate_char_description	1	1	0	2	4
3	supervisor_node	5	6	3	14	13	12	generate_plot	1	1	0	2	4
3	supervisor_tools_node	2	3	0	5	6	12	generate_image_prompts	4	7	0	11	11
3	structure_events	6	4	2	12	10	13	character_introduction	3	5	0	8	8
3	check_chunk_for_events	1	2	1	4	6	13	create_characters	3	7	0	10	10
3	url_finder	1	2	1	4	6	13	create_story	2	4	0	6	6
3	extract_and_categorize	4	8	2	14	11	13	narrator	3	5	0	8	8
3	combine_events	1	3	2	6	9	14	select_top_urls	1	4	0	5	6
4	music_assistant	4	2	2	8	9	14	summarize_articles	1	4	0	5	6
5	ts_writer_node	2	2	2	6	10	15	task_generation_node	4	2	1	7	8
5	ts_reviewer_node	1	2	2	5	9	15	task_dependency_node	2	2	1	5	6
5	tc_writer_node	2	2	2	6	9	15	task_scheduler_node	4	1	3	8	11
5	tc_reviewer_node	1	2	2	5	9	15	task_allocation_node	7	5	4	16	13
6	input_city	1	2	0	3	4	15	risk_assessment_node	8	4	5	17	16
6	input_interests	1	2	0	3	4	15	insight_generation_node	3	2	1	6	7
6	create_itinerary	1	2	0	3	4	15	project_plan_generation	4	1	1	6	7
7	approach_analysis	3	5	0	8	8	16	plan_node	2	2	0	4	5
7	result_approach	3	5	0	8	8	16	research_node	2	2	0	4	5
8	melody_generator	1	2	0	3	4	16	decision_node	2	3	0	5	6
8	harmony_creator	1	2	0	3	4	16	paper_analyzer	4	2	6	12	15
8	rhythm_analyzer	1	2	0	3	4	16	write_abstract	1	1	0	2	4
8	style_adapter	1	2	0	3	4	16	write_introduction	1	2	0	3	4
9	summary_text	2	1	0	3	4	16	write_methods	2	2	0	4	5
9	research_node	2	2	1	5	6	16	write_results	1	1	1	3	6
9	Insta	1	1	0	2	4	16	write_conclusion	3	2	0	5	6
9	Twitter	1	1	0	2	4	16	write_references	1	1	0	2	4
9	Linkedin	1	1	0	2	4	16	aggregator	1	1	0	2	4
9	Blog	1	1	0	2	4	16	critique	3	0	2	5	9
10	check_relevance	3	3	0	6	6	16	paper_reviser	2	2	0	4	5
10	check_grammar	3	3	0	6	6							

Unrealistic test cases. In practice, agents collaborate with each other to perform a task requested by the user. For example, consider a system with three agents, A, B, and C, where A B C. Agent A receives the user's request, while Agent C returns the response to the user. Agents B and C receive the output of the preceding agent. In the proposed approach, we isolate A, B, and C and, for each agent, generate a set of test cases to evaluate its system prompt. Isolating agents may lead to the following issue: the user query inside the test case of an agent may be unrealistic because it is generated from that agent's system prompt. For example, **ARTEMIS** may generate a test case X for agent B, but agent A may have difficulty producing the output required by test case X. Therefore, in future work, we will generate test case X to be compatible with the output format produced by agent A.

Benchmark. Although we select well-known projects, this might not be sufficient to reflect real-world scenario. Some projects are in production and maintained by many developers. However, most agents have scores lower than 4/5. Finding several agents achieving a score of 5 is important to analyze their mechanisms.

Number of user queries. We generate only one user query for each test state z. In the future, we will consider generating multiple user queries that satisfy a test state to better understand the behavior of the agent.

Limitation of internal LLMs. We use the internal LLM Gemini 3 Pro to perform decomposition and test case generation. This LLM could understand the semantics effectively and affordably. If a system prompt is ambiguous, the T/AC/CC decomposition could fail. Although we try to design the prompt thoroughly, the prompt design could still be less effective. In the future, we aim to investigate the prompt design to make the internal LLMs more stable and effective.

Rubric for judge models. We designed a 15 rubric to judge responses. However, other scales and the design of the judge

prompt are not considered.

Failure verification. There are 3,519 failures (Sect. 6.2.2) returned by **ARTEMIS**. Verifying whether these failures are accurate is important. However, we are unable to verify all failures. To enhance the correctness of the results, we conduct two strategies. First, we manually verify about 5% of these failures (approximately 176 pairs). Second, we conduct experiments on Config 2 and Config 3, then compute the difference in scores between two different judges (see *Agreement of judge LLMs* in Sect. 6.2.1). Given the same response, if two judges return approximately the same score, the failure is more confident.

7.2. Construct validity

ARTEMIS uses LLM-as-a-judge to measure the reliability of agents. To make the reliability measurement more convincing, we apply the following solutions.

- *Reduce bias of LLM-as-a-judge*: We acknowledge that LLM-as-a-judge may be biased toward responses generated by the same model. Therefore, we use different judge models and test models. For example, if the test model is GPT, the LLM-as-a-judge is Qwen, and vice versa.
- *Assess a response multiple times*: For each response, we perform the scoring three times to reduce noise. If the response is scored only once and the score is incorrect, the incorrect result may not accurately reflect the reliability of the agent. We therefore take the average of the three scores as the score assigned by LLM-as-a-judge to that response.
- *Assess a response using a rubric*: When scoring a response, we use a clear rubric with a scale from 1 to 5, where 1 corresponds to fail, 3 corresponds to partially correct, and 5 corresponds to fully correct according to user query. In this way, LLM-as-a-judge has a clearer basis for assigning the score.

However, we have not yet included human evaluation to assess the responses. We leave this task for future work.

7.3. Internal threat

The proposed approach uses three LLMs: the internal model, the test model, and the LLM-as-a-judge. The uncertainty of LLMs is one of the factors that can introduce noise into the experimental results. Therefore, we set the temperature of all LLMs to 0 to help the models produce the most stable outputs.

7.4. External threat

To clarify whether **ARTEMIS** is generalizable, we consider the following aspects.

- *Other LLMs*: The experiments only evaluate GPT and Qwen. If other LLMs are used, the accuracy and instability scores may change. This is because the accuracy score of an agent is largely determined by the capability of the underlying LLM.
- *Benchmark*: The experiments are conducted on MASs developed using the LangGraph framework. We have not yet evaluated the effectiveness of **ARTEMIS** on other frameworks, such as CrewAI and AutoGen. Although the benchmark consists of 16 MASs selected from different application domains, we have not covered other application domains. In addition, 5 out of 16 MASs in the benchmark are obtained from production systems published on GitHub. This number may not be sufficient to fully reflect the diverse use cases in real-world applications.

8. Conclusion

This paper presents **ARTEMIS** to test the system prompt of agents. For each agent, **ARTEMIS** decomposes the agent's system prompt into a set of requirements and then generates test cases. A test case consists of a user query and evaluation criteria to evaluate the accuracy score of the response. For each test case, **ARTEMIS** runs the agent multiple times and scores each response multiple times. Based on the scores assigned to the responses, **ARTEMIS** proposes accuracy and instability metrics to evaluate the reliability of the agent. While the accuracy score evaluates whether the response satisfies the user query, the instability score indicates whether the agent returns inconsistent responses when repeatedly run with the same user queries. To demonstrate the effectiveness of **ARTEMIS**, we evaluate **ARTEMIS** on 71 agents from 16 LangGraph systems. The experiments show that the two metrics can detect reliability issues in agents. We also identify the main types of issues existing in system prompts that lead to abnormal agent behavior. Based on these issues, developers have a basis for modifying system prompts to make them more effective. In the future, we will evaluate **ARTEMIS** on more MASs and configurations, as well as test agent improvement based on accuracy and instability scores.

Statements and Declarations

Competing Interests

The authors declare that they have no competing interests.

Funding

The authors did not receive support from any organization for the submitted work.

Ethics Approval

Not applicable.

Data availability

The source code of the **ARTEMIS** prototype, together with the per-agent confidence intervals and the verbatim generator and judge prompt templates, is publicly available at `https://github.com/hype1524/MultiAgentTesting`. The sixteen open-source systems analysed in this study are available at the repositories listed in Table 2.

Author Contributions

Hiep Pham Hoang: Conceptualization, Formal analysis, Investigation, Methodology, Validation, Writing original draft. **Dat Nguyen Trong**: Conceptualization, Formal analysis, Investigation, Methodology, Validation, Writing original draft. **Hoang-Viet Tran**: Conceptualization, Validation, Writing original draft, Writing review & editing. **Duc-Anh Nguyen**: Conceptualization, Project administration, Formal analysis, Investigation, Methodology, Supervision, Resources, Validation, Visualization, Writing original draft, Writing review & editing. **Pham Ngoc Hung**: Conceptualization, Writing original draft, Writing review & editing. All authors reviewed the manuscript.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author(s) used ChatGPT in order to revise English writing. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

References

Atil, B., Aykent, S., Chittams, A., Fu, L., Passonneau, R.J., Radcliffe, E., Rajagopal, G.R., Sloan, A., Tudrej, T., Ture, F., et al., 2024. Non-determinism of" deterministic" llm settings. arXiv preprint arXiv:2408.04667 .

Choi, Y., Baek, J., Hwang, S.J., 2025. System prompt optimization with meta-learning, in: The Thirty-ninth Annual Conference on Neural Information Processing Systems. URL: https://openreview.net/forum?id=IYVknFxsJb.

Confident AI, 2024. DeepEval: The LLM evaluation framework. https://github.com/confident-ai/deepeval. Accessed: 2026-06-13.

Dobslaw, F., Feldt, R., Yoon, J., Yoo, S., 2025. Challenges in testing large language model based software: A faceted taxonomy. ACM Transactions on Software Engineering and Methodology .

Dong, Y., Jiang, X., Qian, J., Wang, T., Zhang, K., Jin, Z., Li, G., 2025. A survey on code generation with llm-based agents. URL: https://arxiv.org/abs/2508.00083, arXiv:2508.00083.

Es, S., James, J., Espinosa Anke, L., Schockaert, S., 2024. RAGAs: Automated evaluation of retrieval augmented generation, in: Aletras, N., De Clercq, O. (Eds.), Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, Association for Computational Linguistics, St. Julians, Malta. pp. 150–158. URL: https://aclanthology.org/2024.eacl-demo.16/, doi:10.18653/v1/2024.eacl-demo.16.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O., Zhang, X., 2024. Large language model based multi-agents: A survey of progress and challenges. arXiv preprint arXiv:2402.01680 .

Hasan, M.M., Li, H., Fallahzadeh, E., Rajbahadur, G.K., Adams, B., Hassan, A.E., 2026. An empirical study of testing practices in open source ai agent frameworks and agentic applications. Empirical Software Engineering 31, 124.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Yau, S., Lin, Z., Zhou, L., et al., 2024. Metagpt: Meta programming for a multi-agent collaborative framework, in: International Conference on Learning Representations, pp. 23247–23275.

Jamshidi, S., Dakhel, A.M., Nafi, K.W., Khomh, F., 2026. Hallucination cascade: Analyzing error propagation in multi-agent llm systems. URL: https://arxiv.org/abs/2606.07937, arXiv:2606.07937.

Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K., 2024. Swe-bench: Can language models resolve real-world github issues?, in: International Conference on Learning Representations, pp. 54107–54157.

Kim, S., Shin, J., Cho, Y., Jang, J., Longpre, S., Lee, H., Yun, S., Shin, S., Kim, S., Thorne, J., Seo, M., 2024. Prometheus: Inducing fine-grained evaluation capability in language models, in: The Twelfth International Conference on Learning Representations. URL: https://openreview.net/forum?id=8euJaTveKw.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., et al., 2024. Agentbench: Evaluating llms as agents, in: International Conference on Learning Representations, pp. 52989–53046.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., Zhu, C., 2023. G-eval: Nlg evaluation using gpt-4 with better human alignment, in: Proceedings of the 2023 conference on empirical methods in natural language processing, pp. 2511–2522.

Luo, J., Zhang, W., Yuan, Y., Zhao, Y., Yang, J., Gu, Y., Wu, B., Chen, B., Qiao, Z., Long, Q., et al., 2025. Large language model agent: A survey on methodology, applications and challenges. arXiv preprint arXiv:2503.21460 .

Ma, W., Yang, C., Kästner, C., 2024. (why) is my prompt getting worse? rethinking regression testing for evolving llm apis, in: Proceedings of the IEEE/ACM 3rd international conference on AI engineering-software engineering for AI, pp. 166–171.

Ma, W., Yang, Y., Hu, Q., Ying, S., Jin, Z., Du, B., Xing, Z., Li, T., Shi, J., Liu, Y., et al., 2025. Rethinking testing for llm applications: Characteristics, challenges, and a lightweight interaction protocol. arXiv preprint arXiv:2508.20737 .

Mialon, G., Fourrier, C., Wolf, T., LeCun, Y., Scialom, T., 2024. Gaia: a benchmark for general ai assistants, in: International Conference on Learning Representations, pp. 9025–9049.

Mohammadi, M., Li, Y., Lo, J., Yip, W., 2025. Evaluation and benchmarking of llm agents: A survey, in: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2, Association for Computing Machinery, New York, NY, USA. p. 61296139. URL: https://doi.org/10.1145/3711896.3736570, doi:10.1145/3711896.3736570.

Neumann, A., Kirsten, E., Zafar, M.B., Singh, J., 2025. Position is power: System prompts as a mechanism of bias in large language models (llms), in: Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency, Association for Computing Machinery, New York, NY, USA. p. 573598. doi:10.1145/3715275.3732038.

Owotogbe, J., 2025. Assessing and enhancing the robustness of llm-based multi-agent systems through chaos engineering, in: 2025 IEEE/ACM 4th International Conference on AI Engineering Software Engineering for AI (CAIN), pp. 250–252. doi:10.1109/CAIN66642.2025.00039.

Pan, M.Z., Cemri, M., Agrawal, L.A., Yang, S., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Ramchandran, K., Klein, D., Gonzalez, J.E., Zaharia, M., Stoica, I., 2025. Why do multiagent systems fail?, in: ICLR 2025 Workshop on Building Trust in Language Models and Applications. URL: https://openreview.net/forum?id=wM521FqPvI.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., et al., 2024. Chatdev: Communicative agents for software development, in: Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers), pp. 15174–15186.

Shamim, I., Singhal, R., 2024. Methodology for quality assurance testing of llm-based multi-agent systems, in: Proceedings of the 4th International Conference on AI-ML Systems, pp. 1–5.

Tian, H., Wang, C., Yang, B., Zhang, L., Liu, Y., 2025. A taxonomy of prompt defects in llm systems. URL: https://arxiv.org/abs/2509.14404, arXiv:2509.14404.

Tran, K.T., Dao, D., Nguyen, M.D., Pham, Q.V., O'Sullivan, B., Nguyen, H.D., 2025. Multi-agent collaboration mechanisms: A survey of llms. arXiv preprint arXiv:2501.06322 .

Uniyal, M., Singh, M., Verbruggen, G., Le, V., Gulwani, S., 2026. Autochecklist: Automated checklist refinement for llm judges, in: Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA. p. 926931. URL: https://doi.org/10.1145/3803437.3805263, doi:10.1145/3803437.3805263.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., et al., 2024. A survey on large language model based autonomous agents. Frontiers of computer science 18, 186345.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al., 2022. Chain-of-thought prompting elicits reasoning in large

language models. Advances in neural information processing systems 35, 24824–24837.

Yagubyan, A., 2026. How consistent are llm agents? measuring behavioral reproducibility in multi-step tool-calling pipelines. URL: https://arxiv.org/abs/2605.28840, arXiv:2605.28840.

Yang, C., Shi, Y., Ma, Q., Liu, M.X., Kaestner, C., Wu, T., 2026. What prompts don't say: Understanding and managing underspecification in LLM prompts, in: Liakata, M., Moreira, V.P., Zhang, J., Jurgens, D. (Eds.), Findings of the Association for Computational Linguistics: ACL 2026, Association for Computational Linguistics, San Diego, California, United States. pp. 9072–9101. URL: https://aclanthology.org/2026.findings-acl.441/, doi:10.18653/v1/2026.findings-acl.441.

Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A., Shmueli-Scheuer, M., 2025. Survey on evaluation of LLM-based agents. arXiv preprint arXiv:2503.16416 URL: https://arxiv.org/abs/2503.16416.

Zamfirescu-Pereira, J., Wong, R.Y., Hartmann, B., Yang, Q., 2023. Why johnny cant prompt: How non-ai experts try (and fail) to design llm prompts, in: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, New York, NY, USA. URL: https://doi.org/10.1145/3544548.3581388, doi:10.1145/3544548.3581388.

Zhang, Z., Li, S., Zhang, Z., Liu, X., Jiang, H., Tang, X., Gao, Y., Li, Z., Wang, H., Tan, Z., Li, Y., Yin, Q., Yin, B., Jiang, M., 2025. IHEval: Evaluating language models on following the instruction hierarchy, in: Chiruzzo, L., Ritter, A., Wang, L. (Eds.), Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Albuquerque, New Mexico. pp. 8374–8398. URL: https://aclanthology.org/2025.naacl-long.425/, doi:10.18653/v1/2025.naacl-long.425.

Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al., 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in neural information processing systems 36, 46595–46623.

Zhu, K., Du, H., Hong, Z., Yang, X., Guo, S., Wang, D.Z., Wang, Z., Qian, C., Tang, X., Ji, H., et al., 2025. Multiagentbench: Evaluating the collaboration and competition of llm agents, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 8580–8622.

Zhu, K., Zhao, Q., Chen, H., Wang, J., Xie, X., 2024. Promptbench: A unified library for evaluation of large language models. Journal of Machine Learning Research 25, 1–22.

Manuscript

Click here to access/download
Supplementary material for on-line publication only
Appendix.pdf

Declaration of interests

☒The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☐The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: